Scaling Biclique Percolation for Community Detection in Large Bipartite Networks

Yue Zeng
Beijing Institute of Technology
bruceez@163.com

Rong-Hua Li
Beijing Institute of Technology
lironghuabit@126.com

Qiangqiang Dai
Beijing Institute of Technology
qiangd66@gmail.com

Guoren Wang
Beijing Institute of Technology
wanggrbit@126.com

***Abstract*—Community detection is a core network-analysis task, and many real-world networks are naturally bipartite. Biclique percolation community (BCPC) detects cohesive bipartite structures by connecting maximal bicliques that share an (α, β)-biclique. Existing BCPC methods are costly because they explicitly traverse the potentially massive maximal biclique adjacency graph (MBAG). We adapt the high-level framework of k-clique percolation community detection on unipartite graphs to the two-sided (α, β) criterion in bipartite graphs. The framework first computes partial-BCPCs using the maximal biclique enumeration tree and then uses them to prune (α, β)-biclique listing, avoiding MBAG traversal. To strengthen this pruning, we develop a cross-subtree partial-BCPC computation method that finds all maximal bicliques sharing a qualifying prefix, including those in other enumeration subtrees. Unlike the previous leaf-level search, it can stop at another qualifying node and reuse its partial-BCPC through a stop-label. Experiments show that our best method achieves a speedup of over 390× compared with existing methods.**

I. Introduction

Community detection aims to uncover closely connected vertex groups and is central to network analysis [1], with applications in social networks [2], biological systems [3], and recommendation systems [4]. Many real-world networks are bipartite graphs $G = (U, V, E)$, where vertices are split into two disjoint sets and edges only connect the two sides, e.g., user–item networks [5] and author–paper networks [6]. This structure motivates specialized community models for bipartite graphs.

The bi̲c̲lique p̲ercolation c̲ommunity (BCPC) is a widely studied approach for detecting cohesive structures in bipartite graphs [7], [8], [9], [10], [11], [12], [13], [14]. A biclique is a complete bipartite subgraph, and a biclique percolation community is defined as a maximal set of bicliques that are connected through *adjacency*. In the (α, β)-biclique percolation community, two bicliques are considered *adjacent* if they share at least α vertices on one side and β on the other (share an (α, β)-biclique).

Example 1. *Figure 1 shows a* $(4, 4)$-BCPC *around Jiawei Han in the* DBLP *author–paper bipartite graph. The dashed boxes* B_1, B_2, B_3 *are three maximal bicliques, none of which individually captures the full community. Since* B_1–B_2 *and* B_2–B_3 *each share a* $(4, 4)$*-biclique, they are linked by biclique adjacency and percolate into a single* BCPC.

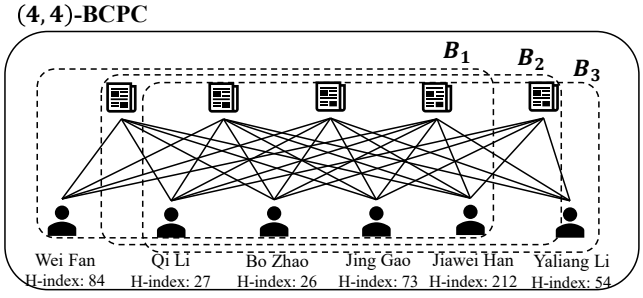


Fig. 1. A $(4, 4)$-BCPC around Jiawei Han in the DBLP author–paper bipartite graph

As illustrated in Example 1, a BCPC is formed by multiple maximal bicliques that *percolate* into a single community without requiring the whole to be a complete biclique, making it a natural relaxation of single-biclique models. The parameters α, β act as cohesion controls: smaller values yield larger, looser communities, while larger values yield tighter, higher-confidence ones.

BCPC has been applied to social networks for analyzing user roles and themes [11], Wikipedia editor–article networks for revealing collaborative editing clusters [8], online-course resource networks for mining learners' access and participation patterns [10], [12], and enterprise security graphs for supporting host–user–application monitoring [9]. BCPC naturally allows entities to participate in multiple communities, which better matches real-world bipartite networks where entities often play multiple roles or appear in multiple functional contexts [11], [8], [10], [9].

Despite its widespread practical utility, the BCPC method faces significant computational challenges due to its high time complexity. To the best of our knowledge, the existing BCPC methods are all based on [7]. Its framework was later implemented in [15]. [13], [14] also study BCPC computation in index-based settings for repeated queries; their index construction still relies on this framework. The basic idea is based on *m̲aximal b̲iclique a̲djacency g̲raph* (MBAG). This method first enumerates all maximal bicliques and then uses them as vertices to construct a maximal biclique adjacency graph. In this graph, there is an edge between two vertices if and only if the overlapping part of the corresponding two maximal bicliques contains an (α, β)-biclique. The connected components in the adjacency graph correspond one-to-one with the (α, β)-BCPC. However, real-world bipartite graphs

may contain a large number of maximal bicliques (up to $2^{n/2}$ theoretically [16]), and the adjacency graph formed by these maximal bicliques could contain a significant number of edges (up to 10^{11}, see Table II). Traversing such a large-scale adjacency graph is highly time-consuming.

To address the cost of traversing the MBAG, we adapt the high-level framework of previous work on unipartite graphs [17] to the two-sided (α, β) criterion in bipartite graphs. We first compute partial-BCPCs from the maximal biclique enumeration tree. When a search-tree prefix contains at least α vertices from U and β vertices from V, all maximal bicliques in its subtree share an (α, β)-biclique and can be pre-merged. We then perform (α, β)-biclique listing to establish the remaining connections without traversing the MBAG. The partial-BCPCs prune a listing branch when the maximal bicliques containing its intermediate biclique are already in the same partial-BCPC.

The basic computation above only merges maximal bicliques within each subtree, although others sharing the same prefix may lie in different subtrees. We therefore develop a new cross-subtree search to compute more complete partial-BCPCs and strengthen the subsequent pruning. Unlike the search in [17], which descends to maximal-clique leaves and processes deeper prefixes to recover missed connections, our search can stop at another qualifying prefix and recover all relevant maximal bicliques in a single search. Stop-labels allow it to reuse the partial-BCPC already computed for a subtree. This yields over 390× speedup over MBAG (e.g., on Youtube with $\alpha = \beta = 2$ in Exp-1).

We summarize our contributions as follows.

Adapted BCPC computation framework. We adapt the high-level framework of previous work [17] to bipartite graphs under the two-sided (α, β) criterion. We introduce (α, β)-nodes in the maximal biclique enumeration tree to compute partial-BCPCs and use them to prune (α, β)-biclique listing, with corresponding bipartite-specific guarantees.

New cross-subtree partial-BCPC computation method. We develop a method that computes partial-BCPCs by finding all maximal bicliques sharing a qualifying prefix, including those in other subtrees. Unlike the previous leaf-level search, it can stop at another qualifying node and reuse its partial-BCPC through a stop-label. A single search therefore recovers the relevant maximal bicliques without repeatedly processing deeper prefixes, strengthening subsequent listing pruning.

Extensive experiments. On 12 real-world datasets, our algorithms achieve large speedups. For example, on Youtube with $\alpha = \beta = 2$, MBAG takes 13,376 seconds, whereas our best algorithm BicliqueP takes only 34 seconds. A case study shows that BCPC captures more meaningful communities than biclique, bitruss, and bicore.

Reproducibility. The source code of this paper is released on https://github.com/bruce1114/bcpc for reproducibility purposes.

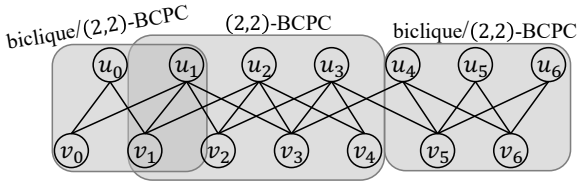


Fig. 2. A bipartite graph G and (α, β)-BCPC ($\alpha = 2, \beta = 2$)

II. PRELIMINARIES

Let $G = (U, V, E)$ be an undirected and unweighted bipartite graph, where U, V are two disjoint sets of vertices and E is a set of edges in form of $(u, v), u \in U, v \in V$. For a vertex $u \in U$, $Nei_G(u)$ is the set of neighbors of u, i.e., $Nei_G(u) = \{v|(u, v) \in E\}$. A bipartite graph $G' = (U', V', E')$ is a subgraph of G if $U' \subseteq U, V' \subseteq V, E' \subseteq E$.

Definition 1 (Biclique)**.** *Given a bipartite graph $G = (U, V, E)$, a biclique is a pair of vertex sets $B = (X, Y)$ where $X \subseteq U, Y \subseteq V$ and $\forall u \in X, v \in Y, (u, v) \in E$.*

For simplicity, we let $B.X = X, B.Y = Y$ if $B = (X, Y)$. For two bicliques B, B', $B \subseteq B'$ indicates that $B.X \subseteq B'.X, B.Y \subseteq B'.Y$. A maximal biclique B is a biclique that there is no other biclique B' satisfying $B \subseteq B'$.

Definition 2 ((α, β)-Biclique Adjacency)**.** *Given two integers α, β and two bicliques B_1, B_2, B_1 and B_2 are (α, β)-adjacent if $|B_1.X \cap B_2.X| \geq \alpha, |B_1.Y \cap B_2.Y| \geq \beta$.*

Definition 3 ((α, β)-Biclique Connectivity)**.** *Given two integers α, β and two bicliques B_1, B_n, B_1 and B_n are (α, β)-connected if there is a sequence of bicliques $B_1, B_2, ..., B_n$ that $\forall i = 1, ..., n-1$, B_i and B_{i+1} are (α, β)-adjacent.*

Definition 4 ((α, β)-Biclique Percolation Community ((α, β)-BCPC))**.** *Given a bipartite graph G and two integers α, β, an (α, β)-biclique percolation community is a set $\mathcal{B}$ of maximal bicliques in G satisfying the following two conditions: (i) any two maximal bicliques in $\mathcal{B}$ are (α, β)-connected; and (ii) $\mathcal{B}$ is maximal, i.e., no maximal biclique outside $\mathcal{B}$ is (α, β)-connected with any maximal biclique in $\mathcal{B}$ (equivalently, $\mathcal{B}$ can not be enlarged by adding any other maximal biclique while keeping condition (i)).*

Example 2. *Let $\alpha = 2, \beta = 2$, Figure 2 illustrates a bipartite graph G and some of the $(2, 2)$-BCPCs. These BCPCs locate on the left, in the middle, and on the right, respectively. The $(2, 2)$-BCPC in the middle is composed of three maximal bicliques: $B_1 = (\{u_1, u_2\}, \{v_1, v_2, v_3\})$, $B_2 = (\{u_1, u_2, u_3\}, \{v_2, v_3\})$, and $B_3 = (\{u_2, u_3\}, \{v_2, v_3, v_4\})$. Among these bicliques, B_1 and B_2 are adjacent because their intersection, $(\{u_1, u_2\}, \{v_2, v_3\})$, satisfies the condition in Definition 2. For the same reason, B_2 and B_3 are also adjacent. Therefore, B_1, B_2, and B_3 can together form a valid $(2, 2)$-BCPC. On the left and right sides of Figure 2 are two maximal bicliques. These maximal bicliques are not adjacent (Definition 2) to any other maximal bicliques, and thus can be regarded as two special BCPCs.*

TABLE I
FREQUENTLY USED NOTATIONS

Notation	Description
$G = (U, V, E)$	bipartite graph with vertex sets U, V and edge set E
$B = (B.X, B.Y)$	a biclique with $B.X \subseteq U$ and $B.Y \subseteq V$
$\mathcal{B}$	a set of maximal bicliques
$\mathcal{P}$	a partial-BCPC
MBAG	maximal biclique adjacency graph
MBE tree	maximal biclique enumeration tree
nd	a node in the MBE tree
$nd.R_U, nd.R_V$	current biclique vertices stored at nd
$nd.C_U, nd.C_V$	candidate vertices for expansion at nd
$nd.X_U, nd.X_V$	exclusion vertices at nd
$nd.RCX_U$	shorthand for $nd.R_U \cup nd.C_U \cup nd.X_U$; similarly for $nd.RCX_V$
(α, β)-node	node where $\lvert R_U \rvert \geq \alpha$ and $\lvert R_V \rvert \geq \beta$ first hold

Problem statement. Given a bipartite graph G and two integers α, β, the goal of our work is to detect all (α, β)-BCPCs (BCPC detection).

A. Maximal Biclique Enumeration

The existing state-of-the-art maximal biclique enumeration methods [18], [19], [16] are all based on the set-enumeration tree structure, where vertices are iteratively traversed to build nodes of the enumeration tree, with each node representing a distinct biclique. In this paper, we adopt the algorithmic framework in [16] to perform maximal biclique enumeration and to design the partial-BCPC based method in Section IV. This is because the method in [16] achieves state-of-the-art performance in maximal biclique enumeration, and furthermore, the framework in [16] is adaptable to a wider range of cohesive subgraph models, making our partial-BCPC based method in Section IV extensible to other cohesive subgraph models in bipartite graphs.

The basic framework [16]. Algorithm 1 presents the basic framework for maximal biclique enumeration in [16]. This framework recursively explores candidate vertex sets to construct the set-enumeration tree, while leveraging exclusion sets to avoid non-maximal bicliques. Algorithm 1 begins by invoking the recursive procedure Enum in Line 1. The parameters of Enum contain the current biclique (R_U, R_V), candidate vertices (C_U, C_V) for expansion, and exclusion vertices (X_U, X_V) that have already been processed. Within Enum, the algorithm first checks the termination condition: $C_U \cup C_V = \emptyset$ (Line 3). If the termination condition is satisfied while $X_U \cup X_V = \emptyset$ (Line 4), it implies that (R_U, R_V) is a maximal biclique. Subsequently, the algorithm iterates over each vertex u in the candidate set C_U (Line 6), calling the Branch procedure to perform branch expansion (Line 7), and moves u from the candidate set C_U to the exclusion set X_U (Line 8). Symmetrically, the algorithm performs the same operation for each vertex v in C_V (Lines 9-11). In the Branch procedure, the algorithm computes the new candidate set $C'_V = C_V \cap Nei_G(u)$ and the new exclusion set $X'_V = X_V \cap Nei_G(u)$, adds u to R_U, and recursively calls the Enum procedure to further explore maximal bicliques within this branch.

Algorithm 1: The basic maximal biclique enumeration framework in [16]

```
   Input: Bipartite graph G = (U, V, E)
   Output: All maximal bicliques in G
1  Enum (∅, ∅, U, V, ∅, ∅);
2  Procedure Enum (R_U, R_V, C_U, C_V, X_U, X_V)
3  if C_U ∪ C_V = ∅ then
4      if X_U ∪ X_V = ∅ then Output (R_U, R_V) as a maximal biclique;
5      return;
6  foreach u ∈ C_U do
7      Branch (R_U, R_V, u, C_U − {u}, C_V, X_U, X_V);
8      C_U ← C_U − {u}; X_U ← X_U ∪ {u};
9  foreach v ∈ C_V do
10     Branch (R_V, R_U, v, C_V − {v}, C_U, X_V, X_U);
11     C_V ← C_V − {v}; X_V ← X_V ∪ {v};
12 Procedure Branch (R_U, R_V, u, C_U, C_V, X_U, X_V)
13 C'_V ← C_V ∩ Nei_G(u); X'_V ← X_V ∩ Nei_G(u);
14 Enum (R_U ∪ {u}, R_V, C_U, C'_V, X_U, X'_V);
```

Pivoting technique [16]. A notable issue with the aforementioned framework is that it generates a significant amount of unnecessary computations that produce non-maximal bicliques. We first make the following observation: for any vertex $u \in U$ in a bipartite graph $G = (U, V)$, and for any maximal biclique $B = (X, Y)$ in the bipartite graph, exactly one of the following two conditions must hold:

1) B contains u (i.e., $u \in X$).
2) B contains a vertex v, where v is a non-neighbor of u in V (i.e., $\exists v \in V - Nei_G(u), v \in Y$).

Based on the above observation, we can conclude that by selecting a pivot vertex u from X_U, C_U, X_V, or C_V, and enumerating only u (if $u \notin X_U, X_V$) and vertices in C_V (or C_U, if $u \in X_V, C_V$) that are not neighbors of u in Lines 6-11 of Algorithm 1, we can ensure the completeness of all maximal bicliques while reducing unnecessary enumeration branches that produce non-maximal bicliques. An effective method for selecting a pivot vertex is to choose the vertex in $X_U \cup C_U$ (or $X_V \cup C_V$) that has the fewest non-neighbor vertices in C_V (or C_U) as the pivot vertex.

Vertex ordering [19]. The efficiency of maximal biclique enumeration algorithms is often influenced by the order in which vertices are processed. [16] applied the 2-hop degree ordering introduced in [19] to their framework. This vertex ordering is only applied at the first level of the recursive search, while the pivoting pruning technique mentioned earlier is utilized in subsequent levels of the recursive search.

Example 3. *Figure 3 illustrates a partial enumeration tree of the maximal biclique enumeration framework [16] on the bipartite graph in Figure 2. The root node initializes its candidate set C_U with the vertices in U ordered by 2-hop degree ordering, while only the branches starting from u_3 and u_2 are shown. For example, after selecting u_3, the node $nd(B_1, h_1)$ adds u_3 to R_U, updates C_V to $Nei_G(u_3)$, and records previously processed vertices u_4, u_5, u_6 in X_U.*

At $nd(B_1, h_1)$, the framework selects v_3 as the pivot and only expands the branch corresponding to v_3, skipping vertices

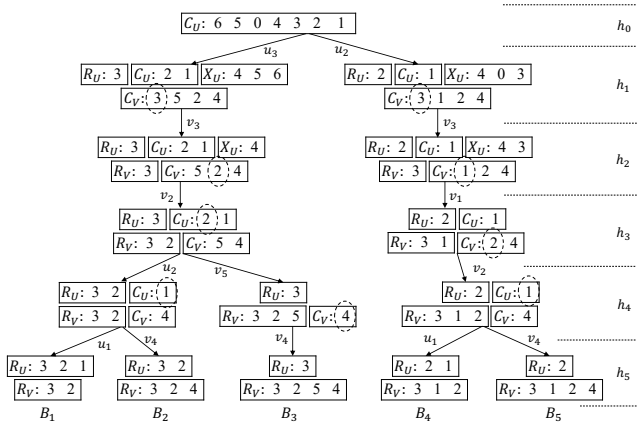


Fig. 3. The partial enumeration tree of the framework in [16] on bipartite graph of Figure 2. Pivot vertices are enclosed in dashed circles. $nd(B_i, h_j)$ represents the tree node at level h_j on the B_i branch of the enumeration tree

Algorithm 2: The (α, β)-biclique enumeration framework in [20]

Input: Bipartite graph $G = (U, V, E)$; Two integers α, β
Output: All (α, β)-bicliques in G

1 Construct 2-hop graph $\vec{H} = (U, \vec{E_h})$ on vertex set U [20];
/* $\vec{H} = (U, \vec{E_h})$: a direct graph, $(u, v) \in \vec{E_h}$ indicates that $u < v$, u, v have common neighbors in V */
2 BicliqueListing $(\vec{H}, \emptyset, V)$;
3 **Procedure** BicliqueListing $(\vec{H}, R_U, C_V)$
4 **if** $|R_U| = \alpha$ **then**
5 **foreach** $R_V \subseteq C_V, |R_V| = \beta$ **do** Output (R_U, R_V) as an (α, β)-biclique ;
6 **return**;
7 **foreach** $u \in \vec{H}$ **do**
8 $C'_V \leftarrow C_V \cap Nei_G(u)$;
9 **if** $|C'_V| < \beta$ *or* $Nei_{\vec{H}}(u) < \alpha - |R_U| - 1$ **then continue**;
10 Let $\vec{H}'$ be induced subgraph of $\vec{H}$ on $Nei_{\vec{H}}(u)$;
11 BicliqueListing $(\vec{H}', R_U \cup \{u\}, C'_V)$;

that cannot lead to new maximal bicliques under the pivoting rule. After v_3 is added to R_V, the candidate and exclusion sets on the U-side are updated by intersecting their old values with $Nei_G(v_3)$. When both candidate sets C_U and C_V become empty, and the exclusion sets X_U and X_V are also empty, the current pair (R_U, R_V) is output as a maximal biclique.

B. (α, β)-Biclique Enumeration

The existing state-of-the-art (α, β)-biclique enumeration method is proposed in [20]. Given a bipartite graph $G = (U, V, E)$, the core idea is to enumerate combinations of vertices in the U set while maintaining their common neighbors in the V set. It also constructs an ordinary directed graph, called the *2-hop graph*, on the vertices of the U set to accelerate the enumeration process.

Algorithm 2 presents the basic framework. It first constructs the *2-hop graph* $\vec{H}$ with U as its vertex set (Line 1), where an edge (u, v) exists if $u < v$ and u, v share at least one common neighbor in set V of the bipartite graph. In Line 4, if R_U already contains α vertices, it is sufficient to enumerate all combinations of β vertices from C_V to obtain the (α, β)-bicliques. In Line 7, the algorithm enumerates vertices u in $\vec{H}$, then in Line 8 updates the candidate set C'_V in V, and proceeds with the next-level enumeration in Line 11.

Algorithm 3: Existing MBAG-based solution

Input: Bipartite graph $G = (U, V, E)$; Two integers α, β
Output: All BCPC in G

1 $\mathcal{B} \leftarrow$ set of all maximal bicliques;
2 $\mathcal{B} \leftarrow \{B | B \in \mathcal{B}, |B.X| \geq \alpha, |B.Y| \geq \beta\}$;
3 $Nei(B) \leftarrow$ set of maximal bicliques that share vertices with B;
4 $UF \leftarrow$ an initial union-find set, i.e., $UF.find(B) = B$ for any input B;
5 **for** $B \in \mathcal{B}$ **do**
6 **for** $B' \in Nei(B)$ **do**
7 **if** $|B.X \cap B'.X| \geq \alpha$ *and* $|B.Y \cap B'.Y| \geq \beta$ **then** $UF.union(B, B')$;
8 **return** UF;

C. Union-Find Set [21]

The Union-Find Set (Disjoint-Set) [21] maintains a collection of disjoint sets supporting two operations: **Find**(i) returns the representative of the set containing i, and **Union**(i, j) merges the sets containing i and j. Both operations run in constant amortized time, with $O(n)$ space for n elements.

III. Existing Solution

The existing BCPC detection method was proposed in [7], implemented in [15] as a toolkit, and used in [13], [14] to build indexes for repeated BCPC queries. To the best of our knowledge, no other work has primarily focused on the algorithmic implementation and improvement of BCPC detection. The basic idea is based on *maximal biclique adjacency graph* (MBAG). This method first enumerates all maximal bicliques and then uses them as vertices to construct a maximal biclique adjacency graph. In this graph, there is an edge between two vertices if and only if the overlapping part of the corresponding two maximal bicliques contains an (α, β)-biclique (satisfying Definition 2). The connected components in the adjacency graph correspond one-to-one with the (α, β)-BCPC. In this method, the (α, β)-biclique connectivity of maximal bicliques in (α, β)-BCPC are maintained by the classic union-find set [21].

Algorithm 3 is the existing MBAG-based algorithm identifying all (α, β)-BCPCs in a bipartite graph $G = (U, V, E)$. It starts by enumerating all maximal bicliques in G and filters them based on size constraints (Lines 1-2). The reason is that for a maximal biclique B, if $|B.X| < \alpha$ or $|B.Y| < \beta$, then B can not form a BCPC with other maximal bicliques. A union-find set UF is initialized in Line 4. The algorithm then iterates over each maximal biclique B and its neighbors $B' \in Nei(B)$, checking if $|B.X \cap B'.X| \geq \alpha, |B.Y \cap B'.Y| \geq \beta$ (Lines 5-7). If the conditions are satisfied, B and B' are merged into the same BCPC by $UF.union(*)$ (Line 7). After processing all maximal bicliques, the union-find set UF represents the final BCPCs in the bipartite graph.

The time complexity of Algorithm 3 is $O(2^{n/2}m + |\mathcal{B}|^2 n)$ (maximal biclique enumeration and MBAG traversal. $n = |U| + |V|, m = |E|$, $\mathcal{B}$ is the set of maximal bicliques). The major drawback of this approach is that the size of the MBAG can be extremely large (up to 10^{11}, see Table II). Consequently, traversing such a massive MBAG to derive all BCPCs is clearly impractical.

IV. BCPC Computation Framework for Bipartite Graphs

As we discussed in Section III, current approach relying on MBAG (Algorithm 3) faces significant challenges due to the vast scale of MBAG. This immense size leads to inefficient traversal operations.

A possible alternative is to avoid materializing the MBAG and directly list the (α, β)-bicliques. For each listed (α, β)-biclique, all maximal bicliques containing it can be merged in a union-find structure. However, the number of (α, β)-bicliques can also be exponential [20], [22].

To avoid both massive MBAG traversal and redundant (α, β)-biclique listing, we adopt the unified framework introduced in previous work [17] on unipartite graphs and instantiate it for bipartite graphs. The framework first exploits the maximal biclique enumeration (MBE) tree to compute partial-BCPCs: for each subtree whose maximal bicliques share a common (α, β)-biclique, it pre-merges these maximal bicliques into a partial-BCPC in a union-find structure. It then performs (α, β)-biclique listing, pruning a search branch whenever the maximal bicliques containing its intermediate biclique already belong to the same partial-BCPC. In this way, partial-BCPCs convert connectivity established during maximal-biclique enumeration into pruning power for the subsequent listing stage.

In the remainder of this section, we first present the basic method for computing partial-BCPCs and then introduce an (α, β)-biclique listing framework for computing the final BCPCs.

A. *Partial*-BCPC *Computation via* MBE *Tree*

This subsection presents how to compute partial-BCPCs using the MBE tree. The key observation is that all maximal bicliques in a subtree of the MBE tree share the biclique represented by its root. If this biclique contains at least α vertices in U and β vertices in V, these maximal bicliques belong to the same BCPC. Based on this observation, we collect these maximal bicliques and merge them into a partial-BCPC. We first illustrate this idea with an example and then formally introduce the MBE tree and the nodes used by the computation algorithm.

Example 4. *Let $\alpha = \beta = 1$. Consider Figure 3, where the subtree rooted at node $nd(B_1, h_2)$ contains three maximal biclique branches: B_1, B_2, and B_3. All three maximal bicliques include both $nd(B_1, h_2).R_U$ and $nd(B_1, h_2).R_V$. Therefore, we can regard B_1, B_2, and B_3 as forming an incomplete* BCPC*, i.e., a partial-*BCPC.

Next, we will formally define partial-BCPC, the maximal biclique enumeration tree and the special prefixes used to compute partial-BCPCs.

Definition 5 (partial-BCPC)**.** *Given a bipartite graph G and two integers α, β, a partial-*BCPC *$\mathcal{P}$ is a set of maximal bicliques satisfying $\mathcal{P} \subseteq \mathcal{B}$, where $\mathcal{B}$ is an (α, β)-*BCPC *in G.*

The distinction from Definition 4 lies in the fact that Definition 5 does not require the set of maximal bicliques in the partial-BCPC to be maximal. This reflects the characteristic of partial-BCPC as an incomplete BCPC.

Definition 6 (Maximal Biclique Enumeration Tree (MBE Tree))**.** *Given a bipartite graph $G = (U, V, E)$. The* MBE *tree $T = (N, \vec{E})$ represents the search space of the maximal biclique enumeration algorithm on G. Here, N denotes the set of nodes in the tree, where each node $nd \in N$ contains a six-tuple $(R_U, R_V, C_U, C_V, X_U, X_V)$, as defined in the basic framework of* MBE *in Section II-A. The set $\vec{E}$ consists of directed edges of the form (nd_i, nd_j, u), where $u \in U$ or $u \in V$. An edge $(nd_i, nd_j, u) \in \vec{E}$ indicates that node nd_j is a child of node nd_i, generated by selecting vertex u from $nd_i.C_U$ or $nd_i.C_V$. A node $nd \in N$ is a leaf node if $nd.C_U \cup nd.C_V = \emptyset$. For a leaf node $nd \in N$, if $nd.X_U \cup nd.X_V = \emptyset$, then $(nd.R_U, nd.R_V)$ is a maximal biclique.*

For simplicity, we use $nd.RCX_U$ and $nd.RCX_V$ to denote $nd.R_U \cup nd.C_U \cup nd.X_U$ and $nd.R_V \cup nd.C_V \cup nd.X_V$, respectively.

Definition 7 ((α, β)-Prefix and (α, β)-Node)**.** *Given a bipartite graph $G = (U, V, E)$ and its* MBE *tree $T = (N, \vec{E})$, let $nd_1 \in N$ be the root node of T, $PF(nd_l) = (nd_1, nd_2, ..., nd_l)$ is the path from nd_1 to nd_l, i.e., $\forall i = 1, 2, ..., l-1, (nd_i, nd_{i+1}, *) \in \vec{E}$. $PF(nd_l)$ is an (α, β)-prefix if and only if $|nd_l.R_U| = \alpha, |nd_{l-1}.R_U| < \alpha, |nd_l.R_V| \geq \beta$ or $|nd_l.R_V| = \beta, |nd_{l-1}.R_V| < \beta, |nd_l.R_U| \geq \alpha$. In this case, nd_l is an (α, β)-node.*

In short, an (α, β)-node nd is the first node that satisfies $|nd.R_U| \geq \alpha$ and $|nd.R_V| \geq \beta$ in all branches where nd resides. For all its ancestor nodes nd_i, it is not possible to simultaneously satisfy $|nd_i.R_U| \geq \alpha$ and $|nd_i.R_V| \geq \beta$.

Focusing on (α, β)-nodes. Ancestors of an (α, β)-node are ignored as they fail to satisfy $|R_U| \geq \alpha$ and $|R_V| \geq \beta$. Descendants of an (α, β)-node, while satisfying $|R_U| \geq \alpha$ and $|R_V| \geq \beta$, are redundant since any maximal biclique branches sharing the prefix corresponding to the descendants must also share the (α, β)-prefix corresponding to the (α, β)-node. Thus, focusing solely on (α, β)-nodes enables efficient computation of partial-BCPCs.

Implementation. Algorithm 4 presents the basic method for partial-BCPC computation. It begins by maximal biclique enumeration and extracting MBE tree $T = (N, \vec{E})$ (Line 1). It then initializes a union-find set UF to manage the partial-BCPCs (Line 2). In Lines 4-7, the algorithm iterates over each (α, β)-node nd in T (Line 4), invoking the ProcessNode function to gather all maximal bicliques under subtree rooted at nd into $\mathcal{P}$ (Line 5). Subsequently, it merges all maximal bicliques in $\mathcal{P}$ into the same partial-BCPC (Line 6). The algorithm finally returns UF, which represents all partial-BCPCs.

The storage of the MBE **tree.** In practice, we do not store all

Algorithm 4: Basic partial-BCPC computation

```
Input: Bipartite graph G = (U, V, E); Two integers α, β
Output: partial-BCPCs
1  Let T = (N, E⃗) be the MBE tree of G, B be the set of maximal bicliques
     in G;
2  Let UF be an initial union-find set;
3  Let P ← ∅;
   /* compute partial-BCPCs                                          */
4  for (α, β)-node nd in T do
5      ProcessNode (nd);
6      for B ∈ P do UF.union(B, P[0]);
7      P ← ∅;
8  return UF;
   /* recursively traverse the MBE tree to gather all
      maximal bicliques under node                                   */
9  Procedure ProcessNode (node)
10 for (node, node′, u) ∈ E⃗ do
11     if node′.C_U ∪ node′.C_V = ∅ then
12         if node′.X_U ∪ node′.X_V = ∅ then
13             P ← P ∪ {(node′.R_U, node′.R_V)};
                   /* (node′.R_U, node′.R_V) is a maximal
                      biclique                                       */
14     else ProcessNode (node′) ;
```

branches of the MBE tree. We first define two types of nodes in the MBE tree.

Definition 8 (Real and Virtual Node)**.** *Given a graph G and its* MBE *tree $T = (N, \vec{E})$, for any node $nd \in N$, if nd is a leaf node and satisfies $nd.X_U = nd.X_V = \emptyset$, then nd is a real node—in this case, $(nd.R_U, nd.R_V)$ is a maximal biclique (see Definition 6). Otherwise, if nd has at least one descendant that is a real node, it is also a real node. If neither of these conditions is satisfied, then nd is a virtual node.*

The ProcessNode procedure in Algorithm 4 only needs to identify the branches composed of real nodes (Lines 11-13), i.e., branches containing maximal bicliques. Thus, in practice, we only store real nodes in the MBE tree.

Analysis. We first analyze the correctness of Algorithm 4, then examine the relationship between the number of partial-BCPCs obtained by Algorithm 4 and the numbers of maximal bicliques as well as BCPCs, and finally analyze the complexity of Algorithm 4.

Theorem 1. *Algorithm 4 correctly computes partial-*BCPC*s.*

Proof. The procedure ProcessNode collects all maximal bicliques within the subtree rooted at the given $node$ into $\mathcal{P}$. Since the parameter is always an (α, β)-node, the maximal bicliques collected in each execution of the procedure must share an (α, β)-biclique. Therefore, these bicliques correctly form a partial-BCPC. □

Theorem 2. *For any two (α, β)-nodes nd_1, nd_2, nd_1 can not be an ancestor or a descendant of nd_2.*

Proof. Based on Definition 7, nd_1 is the first node that satisfies $|nd_1.R_U| \geq \alpha$ and $|nd_1.R_V| \geq \beta$ in all branches where nd_1 resides. The same applies to nd_2. Thus, nd_1, nd_2 are in two different branches, and nd_1 can not be an ancestor or a descendant of nd_2. □

This non-ancestor property is useful for establishing the counting relationship in the next theorem.

Theorem 3. *Given a bipartite graph G, two integers α, β and* MBE *tree $T = (N, \vec{E})$, let $n_{biclique}$ be the number of maximal bicliques, $ND \leftarrow \{nd | nd \in N$, nd is an (α, β)-node, nd is a real node$\}$, $pbcpc$ be the number of partial-*BCPC*s computed by Algorithm 4, $bcpc$ be the number of* BCPC*s, the following inequality holds: $n_{biclique} \geq |ND| = pbcpc \geq bcpc$.*

Proof. Based on Theorem 2, any two (α, β)-nodes in ND can not be in the same branch, and nodes in ND are all real nodes, thus, each node in ND has at least one maximal biclique in its subtree. As a result, $n_{biclique} \geq |ND|$. Since Algorithm 4 connects all maximal bicliques in the subtree of each (α, β)-node in ND into a partial-BCPC, and the maximal bicliques from different subtrees of (α, β)-nodes are distinct, thus, each (α, β)-node in ND corresponds to one partial-BCPC ($|ND| = pbcpc$). Partial-BCPCs are incomplete BCPCs according to Definition 5, thus, $pbcpc \geq bcpc$. □

Theorem 4. *The time and space complexity of Algorithm 4 are $O(2^{n/2}m)$ and $O(n_{biclique}n)$ respectively, where $n = |U| + |V|, m = |E|$, $n_{biclique}$ is the number of maximal bicliques.*

Proof. The time complexity of maximal biclique enumeration is $O(2^{n/2}m)$ [16]. Compared to maximal biclique enumeration, Algorithm 4 introduces additional operations: traversing (α, β)-nodes and collecting maximal biclique branches from the subtrees rooted at these nodes. Since (α, β)-nodes do not reside in the same branch (Theorem 2), the time complexity of these additional operations does not exceed $O(2^{n/2}m)$. As a result, the time complexity of Algorithm 4 is $O(2^{n/2}m)$.

As for the space complexity, the main space consumption in Algorithm 4 comes from the MBE tree and maximal bicliques. Since only the real nodes in the MBE tree are stored, the number of branches in the MBE tree is equal to the number of maximal bicliques, thus, the space complexity of Algorithm 4 is $O(n_{biclique}n)$.

□

B. Pruned (α, β)-Biclique Listing

Recall that our framework enumerates (α, β)-bicliques and connects all maximal bicliques containing each one. The partial-BCPCs computed above allow us to prune a branch when the maximal bicliques containing its current biclique are already known to belong to the same BCPC.

Theorem 5. *Given a bipartite graph G, for a biclique B, if the number of maximal bicliques sharing it is 0, or all those maximal bicliques are in the same* BCPC*, then for any other biclique B' such that $B \subseteq B'$, the number of maximal bicliques sharing B' is also 0, or they are in the same* BCPC*.*

Proof. It is easy to see that any maximal biclique containing B' must also contain B, so the set of maximal bicliques containing B' is a subset of those containing B. Therefore, Theorem 5 can be easily proved. □

Algorithm 5: (α, β)-Biclique based solution with partial-BCPC

Input: Bipartite graph $G = (U, V, E)$; Two integers α, β
Output: All BCPC in G

1 Let UF be union-find set computed by Algorithm 4;
2 $\mathcal{B} \leftarrow$ set of all maximal bicliques in G;
3 $\mathcal{B} \leftarrow \{B | B \in \mathcal{B}, |B.X| \geq \alpha, |B.Y| \geq \beta\}$;
4 Construct 2-hop graph $\vec{H} = (U, \vec{E_h})$ on vertex set U [20];
/* $\vec{H} = (U, \vec{E_h})$: a direct graph, $(u, v) \in \vec{E_h}$ indicates that $u < v$, u, v have common neighbors in V */
5 BicliqueListing $(\vec{H}, \emptyset, V)$;
6 **return** UF;

7 **Procedure** BicliqueListing $(\vec{H}, R_U, C_V)$
8 $RB \leftarrow \{B | R_U \subseteq B.X, B \in \mathcal{B}\}$; /* maximal bicliques sharing R_U */
9 **if** $|\{UF.find(B) | B \in RB\}| \leq 1$ **then return;** /* there is only 1 or 0 UF disjoint set in RB */
10 **if** $|R_U| = \alpha$ **then**
11 **foreach** $R_V \subseteq C_V, |R_V| = \beta$ **do**
12 $RB' \leftarrow \{B | R_V \subseteq B.Y, B \in RB\}$;
13 **if** $|\{UF.find(B) | B \in RB'\}| \leq 1$ **then continue;** /* there is only 1 or 0 UF disjoint set in RB' */
14 Connect maximal bicliques in RB' with $UF.union(*)$;
15 **return**;
16 **foreach** $u \in \vec{H}$ **do**
17 $C'_V \leftarrow C_V \cap Nei_G(u)$;
18 **if** $|C'_V| < \beta$ *or* $Nei_{\vec{H}}(u) < \alpha - |R_U| - 1$ **then continue**;
19 Let $\vec{H}'$ be induced subgraph of $\vec{H}$ on $Nei_{\vec{H}}(u)$;
20 BicliqueListing $(\vec{H}', R_U \cup \{u\}, C'_V)$;

Theorem 5 reveals the core idea of pruning the (α, β)-biclique enumeration space: during the enumeration process, for each intermediate small biclique generated, we can maintain the set of maximal bicliques containing it. If this set is empty or all the maximal bicliques in the set already belong to the same BCPC, then the current enumeration branch can be safely pruned.

Implementation. Algorithm 5 presents the basic framework of the (α, β)-biclique based solution with partial-BCPCs. It is built upon the (α, β)-biclique enumeration framework proposed in [20], where the core idea is to enumerate combinations of vertices in the U set while maintaining their common neighbors in the V set. This framework constructs an ordinary directed graph, called the *2-hop graph*, on the vertices of the U set to accelerate the enumeration process.

In Line 1, Algorithm 5 obtains the partial-BCPCs computed by Algorithm 4. In Lines 2–3, Algorithm 5 prepares the maximal bicliques that need to be connected. In Line 4, it constructs the 2-hop graph according to [20]. Then, in Line 5, it executes the (α, β)-biclique enumeration framework BicliqueListing. In Line 8, BicliqueListing collects maximal bicliques that share $(R_U, \emptyset)$. In Line 9, if the maximal bicliques in RB belong to only one or zero union-find sets, then the entire enumeration branch can be pruned. In Lines 12–14, the algorithm further checks whether the maximal bicliques in RB' that share R_V belong to more than one union-find set. If so, it connects them using $UF.union(*)$.

Analysis. Now we proceed to analyze the correctness and complexity of Algorithm 5.

Theorem 6. *Algorithm 5 correctly computes all* BCPC*s.*

Proof. Without pruning, enumerating all (α, β)-bicliques and connecting all maximal bicliques containing each of them correctly computes all BCPCs. By Theorem 5, the pruned branches cannot produce any new connections between union-find sets. Therefore, pruning does not affect the final result, and Algorithm 5 correctly computes all BCPCs. □

Theorem 7. *The time and space complexity of Algorithm 5 are* $O(2^{n/2}m + (a(\vec{H}_p)^{\alpha-2}|\vec{E}_{hp}||V_p| + \Delta_p)n_{biclique})$ *and* $O(n_{biclique}n)$ *respectively.* $n = |U| + |V|, m = |E|$. *Let* U_p *and* V_p *be the vertex sets shared by any two partial-*BCPC*s,* G_p *be the induced subgraph on* U_p *and* V_p*, and* $\vec{H}_p = (U_p, \vec{E}_{hp})$ *be the 2-hop graph based on* G_p. $a(\vec{H}_p)$ *is the arboricity of* $\vec{H}_p$ *[20],* Δ_p *is the number of* (α, β)*-bicliques in* G_p, $n_{biclique}$ *is the number of maximal bicliques.*

Proof. The time complexity of maximal biclique enumeration [16] and partial-BCPC computation (Theorem 4) are both $O(2^{n/2}m)$. With the partial-BCPCs computed by Algorithm 4, Algorithm 5 prunes the branches that can not connect multiple UF sets based on Theorem 5. This implies that vertices initially contained in only one partial-BCPC need not be considered during the (α, β)-biclique enumeration process. Therefore, Algorithm 5 can be regarded as performing (α, β)-biclique enumeration on G_p. Considering Lines 8-9 and Lines 12-14 of Algorithm 5, its time complexity is $O(2^{n/2}m + (a(\vec{H}_p)^{\alpha-2}|\vec{E}_{hp}||V_p| + \Delta_p)n_{biclique})$.

Similarly, the space complexity of Algorithm 5 is still $O(n_{biclique}n)$. □

V. Cross-Subtree Search for Partial-BCPC Computation

The basic framework in Section IV-A only merges the maximal bicliques located within the subtree of an (α, β)-node nd. It cannot merge all maximal bicliques sharing $(nd.R_U, nd.R_V)$ when some of them are enumerated in other subtrees. These unmerged maximal bicliques may remain in different partial-BCPCs, reducing the pruning effectiveness of the subsequent (α, β)-biclique listing. We therefore introduce a *cross-subtree* search strategy that identifies and merges all maximal bicliques sharing $(nd.R_U, nd.R_V)$ into the same partial-BCPC, including those outside the subtree of nd.

For an (α, β)-node nd, not all maximal bicliques sharing $nd.R_U$ and $nd.R_V$ are located within the subtree rooted at nd. For example, in Figure 4 (a), nd is an (α, β)-node, but B_1 and B_2 are not in the subtree rooted at nd, even though both B_1 and B_2 share $nd.R_U$ and $nd.R_V$. This phenomenon occurs because the vertex ordering and pivoting strategy of maximal biclique enumeration place some vertices in B_1 and B_2 (e.g., u_3) early in the candidate set C_U of the root node, so B_1 and B_2 are enumerated earlier than B_4, B_5 and end up in different subtrees of the MBE tree from nd. Yet, by the definition of partial-BCPC, all such cross-subtree maximal bicliques should still be merged into the same partial-BCPC as those in the subtree of nd. Identifying and merging these cross-subtree maximal bicliques yields a substantially larger

and more powerful partial-BCPC, which we refer to as an *cross-subtree* partial-BCPC.

Challenge 1: Complete cross-subtree search. Previous work on unipartite graphs [17] performs cross-subtree search down to maximal-clique leaves. However, a single search from a prefix cannot recover all maximal cliques containing that prefix, even deeper prefixes are also processed to recover the missing connections. We need to identify every branch that may contain such maximal bicliques without blindly traversing the entire MBE tree.

Challenge 2: Cost of leaf-level search. Even after identifying the relevant branches, descending each branch to maximal-biclique leaves can be costly. The maximum depth of the maximal-clique enumeration trees in our previous datasets is 214 (on Livejournal) [17], whereas the MBE tree on Bibsonomy reaches a depth of 19,883 (Table II, the maximum depth equals the size of the largest maximal biclique, $S_{\max}$). Repeatedly searching down to leaves in such a deep tree would incur substantial overhead.

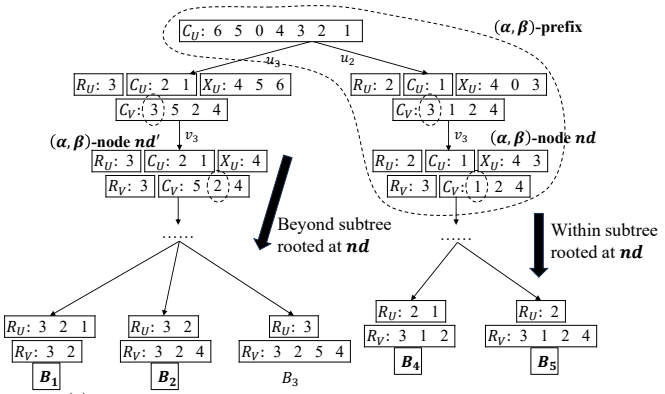
Targeted search via local information. We address the first challenge by leveraging the local R, C, X sets maintained at the (α, β)-node nd. These sets provide a simple necessary condition for deciding whether another branch can generate maximal bicliques sharing the prefix $(nd.R_U, nd.R_V)$. By applying this condition recursively from the root, we can safely exclude irrelevant branches while retaining every branch that may generate a maximal biclique sharing the prefix.

Theorem 8. *Let nd be an (α, β)-node in the* MBE *tree, and let nd' be any other node. If some maximal biclique generated in the subtree rooted at nd' contains $nd.R_U$ and $nd.R_V$, then $nd'.R_U \subseteq nd.RCX_U$ and $nd'.R_V \subseteq nd.RCX_V$.*

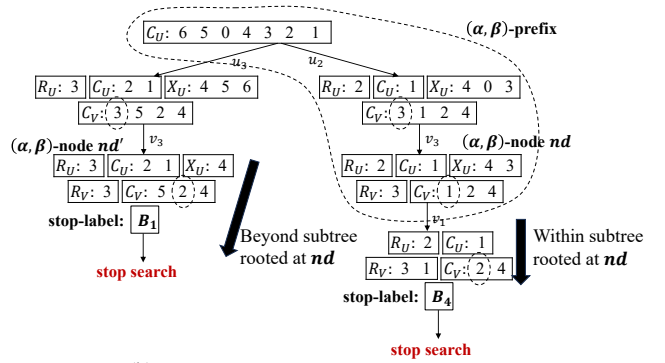
Proof. Let B be a maximal biclique generated in the subtree rooted at nd' and containing $nd.R_U$ and $nd.R_V$. Since B is generated below nd', the current biclique of nd' must be contained in B, i.e., $nd'.R_U \subseteq B.X$ and $nd'.R_V \subseteq B.Y$. By the properties of maximal biclique enumeration, any maximal biclique containing $nd.R_U$ and $nd.R_V$ has all its vertices confined within $nd.RCX_U$ and $nd.RCX_V$ [16]. Thus, $nd'.R_U \subseteq nd.RCX_U$ and $nd'.R_V \subseteq nd.RCX_V$. □

By Theorem 8, any node failing this condition can be safely skipped. Thus, the global traversal is reduced to a targeted search over branches that may contain the prefix $(nd.R_U, nd.R_V)$; the maximal bicliques found there are merged into the same cross-subtree partial-BCPC.

Stop-Label. The targeted search above may still traverse many branches down to leaf nodes. To reduce this overhead, we assign *stop-labels* to real (α, β)-nodes and their immediate real child nodes; each label is an arbitrary maximal biclique in the partial-BCPC containing all maximal bicliques in the corresponding subtree. When searching within the subtree of an (α, β)-node nd, we only collect the stop-labels of its immediate real child nodes, as all maximal bicliques in the



(a) Exploring both branches within and beyond subtree rooted at nd



(b) Using stop-labels to terminate exploration early

Fig. 4. Examples of cross-subtree maximal bicliques. $\alpha = 1, \beta = 1$. B_1, B_2, B_4, B_5 (enclosed in boxes) are maximal bicliques sharing $nd.R_U, nd.R_V$

subtree are guaranteed to be in the same BCPC. In beyond-subtree search, the search can also stop when it reaches and collects the stop-label of another (α, β)-node nd', because maximal bicliques in the subtree of nd' are also guaranteed to be in the same BCPC as those in the subtree of nd (Theorem 9).

Theorem 9. *Let nd be an (α, β)-node in the* MBE *tree. For any other (α, β)-node nd' that satisfies $nd'.R_U \subseteq nd.RCX_U$ and $nd'.R_V \subseteq nd.RCX_V$, every maximal biclique generated in the subtree rooted at nd' belongs to the same* BCPC *as the maximal bicliques in the subtree rooted at nd.*

Proof. By the definition of an (α, β)-node, the sets $(nd.R_U, nd.R_V)$ form a complete bipartite subgraph. Since $nd.RCX_U$ and $nd.RCX_V$ respectively consist of all vertices in G that maintain a complete bipartite relationship with $nd.R_V$ and $nd.R_U$, the condition $nd'.R_U \subseteq nd.RCX_U$ and $nd'.R_V \subseteq nd.RCX_V$ implies that every vertex in $nd'.R_U$ is adjacent to every vertex in $nd.R_V$, and vice versa. Consequently, the union $B^* = (nd.R_U \cup nd'.R_U, nd.R_V \cup nd'.R_V)$ is a biclique. By the property of maximal biclique enumeration, there must exist at least one maximal biclique B_{bridge} in G such that $B^* \subseteq B_{bridge}$. Let B_{nd} be any maximal biclique generated from the subtree of nd, and $B_{nd'}$ be any maximal biclique from the subtree of nd'. B_{nd} and B_{bridge} are connected in the MBAG because they share $(nd.R_U, nd.R_V)$. Similarly, $B_{nd'}$ and B_{bridge} are connected as they share $(nd'.R_U, nd'.R_V)$. Due to this shared adjacency with B_{bridge},

all such maximal bicliques belong to the same BCPC. □

Within-Subtree Stop-Labels. For real child nodes of an (α, β)-node, stop-labels are computed bottom-up. If a child is a leaf, its stop-label is the maximal biclique represented by the leaf; otherwise, the stop-labels of its real child nodes are merged into one partial-BCPC, and any maximal biclique in it becomes the current child's stop-label.

Beyond-Subtree Stop-Labels. If the search for nd reaches another (α, β)-node nd' satisfying Theorem 9 and already having a stop-label, it stops at nd' and directly collects the represented partial-BCPC. All collected stop-labels are finally merged into one partial-BCPC, whose representative maximal biclique becomes the stop-label of nd.

Example 5. *Let $\alpha = 1, \beta = 1$, when processing nd in Figure 4 (b):*

*During the search of subtree rooted at nd, the process terminates upon encountering the stop-label of child node of nd. This stop-label is B_4, with B_5 belonging to the same partial-*BCPC *as B_4.*

During the search beyond the subtree rooted at nd, the search process terminates upon detecting the stop-label of nd'.

*Finally, the two stop-labels (B_1 and B_4) are connected; concurrently, the partial-*BCPC*s they belong to are merged into a larger partial-*BCPC *(including B_1, B_2, B_4, B_5).*

Implementation. Algorithm 6 integrates the above strategies into the basic framework of Algorithm 4 via a post-order traversal (Lines 5, 7). For each (α, β)-node and its descendants, ProcessNode+ (Lines 10, 12) identifies relevant maximal bicliques or existing stop-labels and collects them in $\mathcal{P}$. These are then unified into a single partial-BCPC, with the first element designated as the node's stop-label (Lines 13-15). Specifically, when processing an (α, β)-node (Line 9), ProcessNode+ traverses all its real nodes (Lines 17-19) and initiates targeted recursive search (Line 20) that terminates upon encountering a pre-assigned stop-label (Line 22). Following the MBE enumeration principle, after the branch selecting vertex u (Line 18) has been processed, no subsequent sibling branch can contain u; hence, if $u \in R$, those branches cannot contain the whole target set R and can be skipped (Line 23). For descendant nodes (Line 11), ProcessNode+ either adds the represented maximal biclique (if a leaf, Line 25) or merges the stop-labels of all child nodes into $\mathcal{P}$ (Lines 27-28).

Analysis. We now proceed to analyze the correctness of Algorithm 6.

Theorem 10. *For each (α, β)-node processed in Algorithm 6 (Line 9),* ProcessNode+ *connects all maximal bicliques in the bipartite graph that share $(node.R_U, node.R_V)$ into the same partial-*BCPC*.*

Proof. Consider a sequence of (α, β)-nodes $(nd_1, nd_2, \ldots, nd_n)$, which follow the processing order in Postorder (i.e., post-order traversal). We first prove that during the processing of nd_1, ProcessNode+ connects all maximal bicliques that share $(nd_1.R_U, nd_1.R_V)$ into the same partial-BCPC. Since nd_1 is the first (α, β)-node, it must also be in the first enumerated maximal biclique branch (if nd_1 appears in an earlier branch, then that earlier or an even earlier branch is likely the first one containing a maximal biclique; if nd_1 appears in a later branch, then it can not be the first (α, β)-node). Therefore, all maximal biclique branches that contain $(nd_1.R_U, nd_1.R_V)$ must lie in the subtree rooted at nd_1, and can not appear in any subtree processed before nd_1.

Using mathematical induction, assume that ProcessNode+ connects all maximal bicliques that share $(nd_i.R_U, nd_i.R_V)$ when processing $nd_1, nd_2, ..., nd_i$. Next, we prove the case for $i+1$. For nd_{i+1}, let $R_U = nd_{i+1}.R_U$ and $R_V = nd_{i+1}.R_V$. According to the MBE process [16], the branches containing maximal bicliques with R_U and R_V are either generated before nd_{i+1}, or located in the subtree rooted at nd_{i+1}. For the maximal biclique branches generated before nd_{i+1}, their corresponding (α, β)-node nd_j must satisfy $j < i+1$, so we only need to collect $ST(nd_j)$, as it represents the partial-BCPCs of those maximal bicliques. For the maximal bicliques in the subtree of nd_{i+1}, we only need to collect the stop-labels of the child nodes of nd_{i+1}, because the maximal bicliques under each child node have already been connected into

Algorithm 6: Cross-subtree partial-BCPC computation

```
   Input: Bipartite graph G = (U, V, E); Two integers α, β
   Output: partial-BCPCs
1  Let T = (N, Ē) be the MBE tree of G, B be the set of maximal bicliques
     in G;
2  Let UF be an initial union-find set;
3  Let P ← ∅, root ← root node of T;
4  Let ST(nd), nd ∈ N be the stop-label of nd;
   /* compute partial-BCPCs                                          */
5  Postorder (root);
6  return UF;
7  Procedure Postorder (node)
8  foreach child of node, node', in creation order of node' do Postorder
     (node') ;
9  if node is an (α, β)-node then
10     ProcessNode+
       (root, node.RCX_U ∪ node.RCX_V, node.R_U ∪ node.R_V);
11 else if node is a real node and a descendant of an (α, β)-node then
12     ProcessNode+ (node);
13 if P is not empty then
14     for B ∈ P do UF.union(B, P[0]);
15     ST(node) ← P[0], P ← ∅;
16 Procedure ProcessNode+ (node, RCX, R)
17 foreach child of node, node', in creation order of node' do
18     Let (node, node', u) ∈ Ē;
19     if node' is a real node then
           /* targeted search: only maximal bicliques
              selecting vertices in RCX may contain the
              target prefix R                                        */
20         if u ∈ RCX then
21             if ST(node') = None then ProcessNode+
                 (node', RCX, R) ;
22             else P ← P ∪ {ST(node')} ;
23     if u ∈ R then break;
24 Procedure ProcessNode+ (node)
25 if node is a leaf node then P ← P ∪ {(node.R_U, node.R_V)};
26 else
27     foreach node', node' is a child of node and a real node do
28         P ← P ∪ {ST(node')}
```

partial-BCPCs (since the traversal is post-order). Therefore, the case for nd_{i+1} is proved. □

Theorem 11. *Given a bipartite graph G, two integers α, β, let pbcpc be the number of within-subtree partial-*BCPC*s computed by Algorithm 4, pbcpc+ be the number of cross-subtree partial-*BCPC*s computed by Algorithm 6, bcpc be the number of* BCPC*s, the following inequality holds: $pbcpc \geq pbcpc+ \geq bcpc$.*

Proof. We only need to prove $pbcpc \geq pbcpc+$. When processing an (α, β)-node, Algorithm 6 not only connects maximal bicliques in the subtree rooted at that node, but also further connects maximal bicliques that were enumerated in earlier branches and share R_U, R_V of the (α, β)-node. Moreover, the ProcessNode+ procedure in Algorithm 6 also processes (α, β)-nodes that belong to virtual nodes. Therefore, $pbcpc \geq pbcpc+$. □

Theorem 12. *The time and space complexity of Algorithm 6 are $O(2^{n/2}mn^d)$ and $O(n_{biclique}n)$ respectively, where $n = |U| + |V|, m = |E|$, d is the maximum depth of real (α, β)-nodes (Definition 8), $n_{biclique}$ is the number of maximal bicliques.*

Proof. The time complexity of maximal biclique enumeration is $O(2^{n/2}m)$ [16]. Unlike Algorithm 4, Algorithm 6 handles not only (α, β)-nodes (Lines 9-10) but also their child nodes (Lines 11-12). For processing the child nodes of (α, β)-nodes, Algorithm 6 only needs to traverse one layer of their child nodes, resulting in an overall time complexity not exceeding $O(2^{n/2}m)$. Regarding the processing of (α, β)-nodes, let x be the total number of such nodes (including both real nodes and virtual nodes). When processing each of these nodes (Line 10), the procedure ProcessNode+ will access at most all real (α, β)-nodes. Since the number of child nodes of each node in the MBE tree does not exceed n, the total time complexity of the above process is $O(xn^d) = O(2^{n/2}mn^d)$. As a result, the time complexity of Algorithm 6 is $O(2^{n/2}mn^d)$.

Similar to Algorithm 4, the space complexity of Algorithm 6 is also $n_{biclique}n$. □

Overall Complexity Discussion. Combining cross-subtree partial-BCPC computation in Algorithm 6 with the (α, β)-biclique listing in Algorithm 5 gives a time complexity of $O(2^{n/2}mn^d + (a(\vec{H}_{p^+})^{\alpha-2}|\vec{E}_{h,p^+}||V_{p^+}| + \Delta_{p^+})n_{biclique})$. Here, G_{p^+}, $\vec{H}_{p^+}$, and Δ_{p^+} are defined as G_p, $\vec{H}_p$, and Δ_p in Theorem 7, but using cross-subtree partial-BCPCs. The bound follows by adding the cost of cross-subtree partial-BCPC computation (Theorem 12) to the cost of the subsequent pruned listing.

A BCPC is built from bicliques, and biclique-related problems such as maximal biclique enumeration [18], [19], [16] and (α, β)-biclique enumeration [20], [22] are well-known to be exponential in the worst case, since the number of such bicliques in a bipartite graph can itself be exponential. Exact BCPC detection therefore inherits an exponential worst-case complexity, and our algorithm is no exception. We note, however, that this exponential bound mostly reflects the theoretical worst case rather than the behavior on real-world bipartite graphs in practice [19], [16], [20].

VI. Experiments

A. Experimental Setup

Datasets. Table II presents 12 real-world bipartite graph datasets. The first 10 datasets are downloaded from http://konect.cc/. The two additional datasets, Amazon − Books and Amazon − Beauty, correspond to amazon-books and amazon-beauty-and-personal-care, respectively, from https://huggingface.co/datasets/McAuley-Lab/Amazon-Reviews-2023.

Algorithms. We first implement the state-of-the-art algorithm in [7], [15], [14] as MBAG.

- MBAG (Algorithm 3) is directly based on the maximal biclique adjacency graph (also abbreviated as MBAG). It first enumerates all maximal bicliques, then traverses maximal biclique adjacency graph to compute the BCPC.

We then use partial-BCPCs to optimize the MBAG method and propose several (α, β)-biclique based solutions.

- PBCPC is an improved version of MBAG. It first computes within-subtree partial-BCPCs using Algorithm 4, then merges the maximal bicliques in each partial-BCPC into one MBAG vertex.
- PBCPC+ is an improved version of MBAG. It first computes cross-subtree partial-BCPCs using Algorithm 6, then merges the maximal bicliques in each partial-BCPC into one MBAG vertex, further reducing the number of vertices.
- Biclique is the basic solution based on (α, β)-biclique enumeration, without pruning. Its core idea is to enumerate all (α, β)-bicliques, and for each such biclique, connect all maximal bicliques that contain it in order to obtain the BCPC.
- BicliqueM is an improved version of Biclique. Without computing partial-BCPCs in advance, it prunes (α, β)-biclique enumeration branches by checking whether the maximal bicliques associated with the currently enumerated biclique already belong to the same BCPC.
- BicliqueP combines the (α, β)-biclique listing in Algorithm 5 with the cross-subtree partial-BCPC computation in Algorithm 6, enabling stronger pruning.

Parameters. There are two parameters used in the experiments: α and β. Their default values are both set to 4, and the possible values they can take are 2, 4, 6, and 8. Recall that α and β control the community cohesion: smaller values yield larger and looser communities, while larger values yield tighter and higher-confidence ones.

All the algorithms are implemented in C++. The experiments are conducted on a system running Ubuntu 20.04.4 LTS with an AMD Ryzen 3990X 2.2GHz CPU and 256GB of memory.

TABLE II

DATASETS. THE COLUMNS U, V AND E DESCRIBE THE MEANING OF THE TWO VERTEX SETS AND THE EDGE SET IN EACH BIPARTITE GRAPH; $\overline{d}_U$ IS THE AVERAGE DEGREE OF VERTICES IN U, $\overline{d}_V$ IS THE AVERAGE DEGREE OF VERTICES IN V, N_m IS THE NUMBER OF MAXIMAL BICLIQUES, $S_{\max}$ IS THE MAXIMUM NUMBER OF VERTICES IN A MAXIMAL BICLIQUE, N_o IS THE NUMBER OF EDGES IN MAXIMAL BICLIQUE ADJACENT GRAPH, K=10^3, M=10^6, B=10^9

Datasets	U	V	E	$\|U\|$	$\|V\|$	$\|E\|$	$\overline{d}_U$	$\overline{d}_V$	N_m	$S_{\max}$	N_o
Youtube	user	group	membership	94,238	30,087	293,360	3.11	9.75	1,769,331	1,319	2B
Bookcrossing	user	book	rating	77,802	185,955	433,652	5.57	2.33	155,391	378	1B
Bibsonomy	tag	publication	assignment	204,674	767,448	2,499,057	12.48	3.32	2,132,826	19,883	26B
Citeseer	author	paper	authorship	105,353	181,395	512,267	4.86	2.82	54,083	107	962K
Stackoverflow	user	post	favorite	545,195	96,678	1,301,942	2.39	13.47	2,922,148	1,121	26B
Twitter	user	hashtag	usage	175,214	530,418	1,890,661	10.79	3.56	5,102,542	5,159	13B
Imdb	person	work	association	685,568	186,414	2,715,604	3.96	14.57	1,809,175	320	1B
Actor2	actor	movie	appearance	303,617	896,302	3,782,463	12.46	4.22	3,761,666	389	2B
Amazon	user	product	rating	2,146,057	1,230,915	5,743,258	2.68	4.67	5,702,211	661	108B
DBLP	author	paper	authorship	1,953,085	5,624,219	12,282,059	6.29	2.18	1,281,831	428	85M
Amazon – Books	user	product	rating	10,297,355	4,446,065	29,139,329	2.83	6.55	19,062,947	2,558	109B
Amazon – Beauty	user	product	rating	11,325,787	1,028,731	23,591,312	2.08	22.93	2,998,164	787	5B

B. Experimental Results

Exp-1: Efficiency of various algorithms. In this experiment, we evaluated the performance of all algorithms across all datasets, and the results are presented in Table III and Table IV.

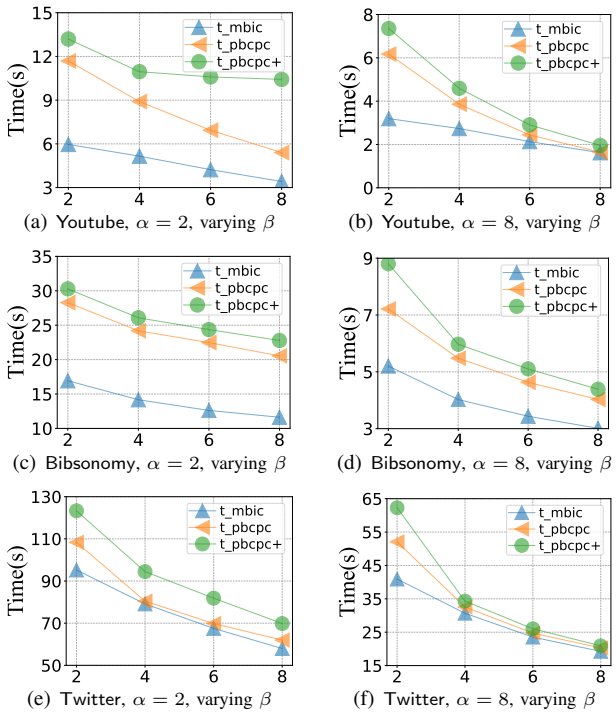
Overall, MBAG and PBCPC algorithms perform the worst in terms of runtime across most datasets and α, β. This is mainly because the MBAG algorithm traverses the original maximal biclique adjacency graph (MBAG), which is often very large (see Table II). Although the PBCPC algorithm attempts to reduce the size of the MBAG by computing partial-BCPCs, these partial-BCPCs are not sufficiently comprehensive, leading to limited pruning effectiveness on the MBAG (we will explore the effectiveness of these partial-BCPCs in Exp-3).

PBCPC+ performs slightly better than MBAG and PBCPC, especially when α and β are small. For instance, on dataset Youtube, when $\alpha = \beta = 2$, PBCPC+ outperforms MBAG and PBCPC by more than one order of magnitude. Small α, β allow more maximal bicliques to satisfy the size threshold and become adjacent, making the MBAG larger. Thus, the MBAG compression brought by cross-subtree partial-BCPCs in PBCPC+ becomes more effective.

Biclique achieves relatively good performance only when both α and β are small—for example, on datasets Youtube, Bookcrossing, and DBLP with $\alpha = \beta = 2$. When α or β increases, the performance of Biclique drops sharply. This is because the number of (α, β)-bicliques grows exponentially as α and β increase.

BicliqueM and BicliqueP are the two optimal algorithms in almost all cases, and they can outperform MBAG by nearly three orders of magnitude. For example, on dataset Youtube, when $\alpha = \beta = 2$, BicliqueP only takes 34 seconds, while MBAG requires 13,376 seconds. Furthermore, BicliqueP performs better than BicliqueM in most cases. This is consistent with the stronger pruning enabled by cross-subtree partial-BCPCs in BicliqueP compared with maximal biclique based pruning.

Exp-2: Overhead of computing partial-BCPC. This experiment evaluates the overhead of computing partial-BCPCs (with maximal biclique enumeration included, which is a necessary step for computing partial-BCPC). Specifically, we compare the time spent on computing partial-BCPCs in Algorithm 4 (t_pbcpc) and Algorithm 6 (t_pbcpc+), and compare both against the time spent on maximal biclique enumeration only. The experimental results are shown in Figure 5.



(a) Youtube, $\alpha = 2$, varying β (b) Youtube, $\alpha = 8$, varying β
(c) Bibsonomy, $\alpha = 2$, varying β (d) Bibsonomy, $\alpha = 8$, varying β
(e) Twitter, $\alpha = 2$, varying β (f) Twitter, $\alpha = 8$, varying β

Fig. 5. Time spent on maximal biclique enumeration (t_mbic) and computing partial-BCPCs in Algorithm 4 (t_pbcpc) and Algorithm 6 (t_pbcpc+); the latter two include maximal biclique enumeration

As shown in Figure 5, although computing partial-BCPCs introduces additional overhead on top of maximal biclique enumeration, the added cost is manageable. On these datasets, t_pbcpc and t_pbcpc+ are slightly higher than the time spent on maximal biclique enumeration alone (t_mbic).

Exp-3: Numbers of maximal bicliques and partial-BCPCs. Table V compares the numbers of maximal bicliques (n_{MB}), within-subtree partial-BCPCs (n_{PBCPC}) computed by Algorithm 4, and cross-subtree partial-BCPCs (n_{PBCPC^+}) computed by Algorithm 6 on all 12 datasets. In every setting, $n_{\mathsf{MB}} \geq n_{\mathsf{PBCPC}} \geq n_{\mathsf{PBCPC}^+}$, showing the additional reduction from cross-subtree merging. For example, on Youtube with $\alpha = \beta = 2$, the three numbers decrease from 1,769,331

TABLE III

RUNNING TIME (SECOND) OF ALL ALGORITHMS ON THE FIRST 10 DATASETS. THE BEST AND SUB-BEST RESULTS ARE BOLD AND UNDERLINED RESPECTIVELY. "-" DENOTES OUT OF TIME ($\geq$ 1 DAYS OR 86,400 SECONDS)

Datasets		Youtube				Bookcrossing				Bibsonomy				Citeseer				Stackoverflow			
α	β / Algo	2	4	6	8	2	4	6	8	2	4	6	8	2	4	6	8	2	4	6	8
2	MBAG	13,376	11,492	6,072	1,902	35	2	0.9	0.7	9,058	8,096	6,980	5,505	1	0.8	0.7	0.6	32,598	17,967	4,287	844
	PBCPC	4,651	7,283	6,942	3,207	42	3	1	1	5,173	5,315	5,636	5,188	1	0.8	0.7	0.6	30,051	25,220	8,811	2,177
	PBCPC+	197	1,914	2,787	1,484	20	3	1	0.8	356	892	1,362	1,736	1	0.9	0.8	0.6	2,564	7,144	3,467	987
	Biclique	135	358	2,995	-	3	26	-	-	193	-	-	-	1	1	64	6,999	2,038	3,118	-	-
	BicliqueM	59	70	43	22	**2**	**1**	**0.8**	**0.6**	54	52	52	52	1	**0.8**	**0.7**	0.6	486	208	**119**	**80**
	BicliqueP	**34**	**43**	**31**	**20**	3	2	1	1	**48**	**40**	**39**	**37**	**1**	0.9	0.8	**0.6**	**339**	**159**	121	103
4	MBAG	13,113	11,358	5,941	1,793	3	**0.7**	**0.5**	**0.4**	8,663	7,755	6,734	5,079	0.1	0.08	0.07	0.06	27,582	14,496	3,308	544
	PBCPC	7,748	9,568	7,802	3,197	4	0.9	0.6	0.5	5,439	6,427	6,254	5,647	0.1	0.09	0.08	0.06	37,632	25,336	7,496	1,476
	PBCPC+	1,450	2,964	3,404	1,618	4	0.8	0.6	0.5	1,010	1,474	1,969	2,267	0.1	**0.08**	0.07	0.06	13,222	9,955	3,857	864
	Biclique	3,754	5,445	5,522	2,877	2	1	0.8	0.6	7,946	-	-	-	0.1	0.1	0.1	0.3	1,210	5,446	748	11,595
	BicliqueM	54	235	140	50	1	0.8	0.6	0.5	106	100	89	82	0.1	0.09	**0.07**	**0.06**	306	539	199	56
	BicliqueP	**28**	**66**	**49**	**24**	**1**	0.9	0.6	0.5	**53**	**46**	**44**	**39**	**0.1**	0.09	0.08	0.07	**190**	**159**	**76**	**34**
6	MBAG	10,936	9,409	4,635	1,312	1	0.6	0.5	0.4	7,280	6,859	5,765	4,367	**0.03**	**0.03**	**0.03**	**0.03**	9,142	4,150	561	45
	PBCPC	7,736	8,370	5,855	2,080	1	0.7	0.5	0.4	5,219	6,098	6,169	5,605	0.05	0.04	0.04	0.04	12,482	6,893	1,134	89
	PBCPC+	2,500	3,622	3,133	1,121	1	0.7	0.5	0.4	1,572	2,050	2,429	2,583	0.04	0.03	0.04	0.03	5,758	3,833	775	76
	Biclique	-	3,289	34,084	3,352	36	0.9	0.5	0.4	9,143	-	-	-	0.07	0.04	0.04	0.04	-	1,764	2,724	26
	BicliqueM	44	165	178	35	1	**0.6**	0.5	0.4	72	138	124	107	0.04	0.04	0.04	0.04	90	131	40	12
	BicliqueP	**24**	**57**	**56**	**16**	**1**	0.7	**0.5**	**0.4**	**39**	**72**	**57**	**54**	0.05	0.04	0.04	0.04	**75**	**72**	**22**	**10**
8	MBAG	7,236	6,102	2,832	744	1	**0.5**	0.4	0.3	5,410	5,144	4,299	3,133	0.03	0.03	0.03	0.03	1,377	398	21	7
	PBCPC	5,585	5,707	3,596	1,241	1	0.6	0.4	0.3	4,829	5,309	5,229	4,508	0.03	0.03	0.03	0.03	1,721	592	32	6
	PBCPC+	2,327	2,878	1,991	713	1	0.6	0.4	0.3	1,735	2,012	2,291	2,268	0.03	0.03	0.03	0.03	1,110	447	29	7
	Biclique	-	-	4,728	8,555	2,315	6	0.4	0.3	-	-	-	-	0.09	0.04	0.04	0.04	-	77	20	14
	BicliqueM	33	117	101	38	1	0.6	0.4	0.3	50	137	178	103	**0.03**	**0.03**	**0.03**	0.03	30	22	9	7
	BicliqueP	**19**	**45**	**33**	**13**	**1**	0.6	**0.4**	**0.3**	**32**	**58**	**85**	**56**	0.04	0.04	0.04	**0.03**	**30**	**16**	**9**	**6**

Datasets		Twitter				Imdb				Actor2				Amazon				DBLP			
α	β / Algo	2	4	6	8	2	4	6	8	2	4	6	8	2	4	6	8	2	4	6	8
2	MBAG	11,652	5,290	1,817	657	859	706	622	409	7,303	6,650	5,217	3,502	16,282	3,974	580	137	18	**12**	10	9
	PBCPC	47,374	24,237	6,055	2,335	323	188	293	418	1,820	2,285	2,377	2,041	20,703	6,687	1,297	342	28	19	15	13
	PBCPC+	10,580	11,843	3,162	1,337	102	58	90	160	266	524	772	780	2,413	3,067	706	228	23	15	12	10
	Biclique	685	5,339	4,584	-	55	147	58,284	-	190	875	39,764	-	964	727	-	-	20	176	-	-
	BicliqueM	**510**	777	208	108	**35**	**26**	**23**	**21**	88	56	48	47	618	**157**	**79**	**55**	**17**	13	**10**	**9**
	BicliqueP	518	**402**	**152**	**94**	42	33	28	25	**79**	**51**	**44**	**39**	**590**	176	108	82	25	19	16	14
4	MBAG	4,339	2,737	1,236	457	762	687	614	403	7,107	6,586	5,178	3,468	7,856	1,720	227	57	**7**	5	4	3
	PBCPC	11,436	6,764	3,176	1,482	408	355	446	514	1,622	2,251	2,462	2,125	9,234	2,496	427	100	8	5	4	3
	PBCPC+	6,282	4,237	2,004	1,001	99	74	121	185	201	516	794	840	4,219	1,508	304	78	8	5	4	4
	Biclique	-	12,028	20,544	-	92	1,805	1,335	25,431	626	4,996	29,168	-	534	2,687	474	9,676	12	7	29	1,834
	BicliqueM	134	129	94	87	**22**	21	26	21	57	131	51	44	303	161	61	**39**	8	5	4	3
	BicliqueP	**134**	**100**	**69**	**64**	23	**18**	**16**	**15**	**47**	**48**	**34**	**31**	**280**	**111**	**61**	42	9	**5**	**4**	**3**
6	MBAG	2,184	1,596	891	298	623	600	536	340	6,859	6,376	5,085	3,343	1,576	349	63	32	3	2	2	2
	PBCPC	3,124	2,404	1,677	690	557	551	605	568	1,988	2,585	2,583	2,270	1,669	499	97	44	3	2	2	2
	PBCPC+	2,666	1,469	1,073	506	169	164	204	237	326	641	933	955	1,035	331	71	36	3	3	2	2
	Biclique	-	-	-	27,608	1,748	835	11,021	10,968	-	22,912	49,668	-	33,212	1,595	525	806	10,632	6	17	30
	BicliqueM	**90**	81	68	52	**17**	17	22	23	51	121	62	52	**171**	80	**44**	**32**	3	2	2	2
	BicliqueP	95	**58**	**51**	**41**	19	**14**	**14**	**13**	**40**	**43**	**38**	**32**	213	**79**	47	33	**3**	**2**	**2**	**2**
8	MBAG	1,482	1,166	651	210	400	388	340	187	6,381	5,838	4,529	2,958	445	104	41	28	1	1	1	1
	PBCPC	1,479	1,297	994	369	440	437	450	331	2,631	2,922	2,867	2,398	539	143	54	34	1	1	1	2
	PBCPC+	880	729	669	271	187	186	209	193	625	894	1,093	1,023	338	101	45	31	2	1	1	1
	Biclique	-	-	-	-	-	2,375	1,985	11,003	-	-	-	-	-	29,402	3,606	355	-	25	61	93
	BicliqueM	**74**	65	53	36	**15**	13	16	16	46	100	151	66	**143**	**59**	**38**	**28**	**1**	1	1	1
	BicliqueP	79	**45**	**36**	**28**	16	**12**	**12**	**11**	**35**	**38**	**46**	**36**	188	64	40	29	2	**1**	**1**	**1**

to 292,951 and then to 48,346. Across the 23 settings with nonzero denominators, $n_{\mathsf{PBCPC^+}}/n_{\mathsf{MB}}$ is below 50% in 17 cases, and $n_{\mathsf{PBCPC^+}}/n_{\mathsf{PBCPC}}$ is below 50% in 16 cases. These reductions allow PBCPC and PBCPC+ to use fewer MBAG vertices and strengthen the listing pruning in BicliqueP.

Exp-4: Scalability. This experiment explores the scalability of our best algorithm, BicliqueP. Table VI presents the performance of BicliqueP on 4 datasets. Specifically, we sampled 20%, 40%, 60%, and 80% of the edges from each dataset respectively. It can be observed that as the dataset size increases, the growth of running time of BicliqueP is manageable, demonstrating that BicliqueP has excellent scalability.

Exp-5: Memory. Table VII presents the memory usage of all algorithms on 4 datasets. It can be observed that across all datasets, the 3 algorithms—MBAG, Biclique, and BicliqueM —consistently consume the least memory. This is because these algorithms do not require storing the MBE tree. However, although our best algorithm, BicliqueP, also stores the MBE tree, its memory consumption remains manageable.

Exp-6: Case study. We conduct a case study on the DBLP dataset. This dataset includes papers from major journals (TODS, TOIS, TKDE, VLDBJ) and conferences (SIGMOD, SIGKDD, ICDE, SIGIR, VLDB) in the database field, where vertices represent papers and authors respectively, and an edge (u, v) indicates that v is one of the authors of u. We compare (α, β)-BCPC with maximum biclique, k-bitruss [23], and (α, β)-core [24].

Given a bipartite graph $G = (U, V, E)$, (α, β)-core of G is a maximal subgraph of G, $G_1 = (U_1, V_1, E_1)$, that $\forall u \in U_1, |Nei_{G_1}(u)| \geq \alpha, \forall v \in V_1, |Nei_{G_1}(v)| \geq \beta$. k-bitruss is a maximal subgraph of G, $G_2 = (U_2, V_2, E_2)$, that each edge in E_2 is contained in at least k butterflies (butterfly is a subgraph

TABLE IV
RUNNING TIME (SECOND) OF ALL ALGORITHMS ON Amazon − Books AND Amazon − Beauty. THE BEST AND SUB-BEST RESULTS ARE BOLD AND UNDERLINED, RESPECTIVELY

Datasets		Amazon − Books				Amazon − Beauty			
α	β / Algo	2	4	6	8	2	4	6	8
2	MBAG	-	-	-	55,130	1,027	344	177	**108**
	PBCPC	-	-	-	66,115	1,885	660	291	161
	PBCPC+	5,506	12,713	25,487	15,686	1,101	467	258	165
	Biclique	3,094	-	-	-	662	612	-	-
	BicliqueM	2,473	1,742	961	**637**	**489**	**235**	**152**	112
	BicliqueP	**2,050**	**1,403**	**920**	661	718	372	235	147
4	MBAG	-	-	-	54,152	534	207	**105**	**70**
	PBCPC	-	-	-	65,987	802	350	166	94
	PBCPC+	10,825	17,437	32,165	17,618	636	292	158	95
	Biclique	-	-	-	-	2,303	249	114	96
	BicliqueM	1,230	3,304	1,605	827	**317**	**170**	109	72
	BicliqueP	**1,106**	**1,167**	**827**	**538**	521	268	148	92
6	MBAG	-	-	-	50,540	331	143	**82**	**57**
	PBCPC	-	-	-	-	500	239	126	77
	PBCPC+	17,665	26,755	32,456	18,010	470	227	128	76
	Biclique	-	-	-	-	-	163	96	57
	BicliqueM	943	2,545	4,864	1,113	**269**	**137**	86	58
	BicliqueP	**868**	**976**	**1,211**	**555**	441	230	124	74
8	MBAG	-	-	-	40,905	269	**116**	72	50
	PBCPC	-	-	-	50,632	411	198	106	63
	PBCPC+	21,212	26,989	29,802	15,724	404	195	106	63
	Biclique	-	-	-	-	-	164	**71**	50
	BicliqueM	794	1,967	3,695	1,199	**247**	118	74	**49**
	BicliqueP	**739**	**836**	**1,039**	**738**	402	195	105	63

TABLE V
NUMBERS OF MAXIMAL BICLIQUES AND PARTIAL-BCPCS ON ALL 12 DATASETS. n_{MB} DENOTES THE NUMBER OF MAXIMAL BICLIQUES, n_{PBCPC} THE NUMBER OF WITHIN-SUBTREE PARTIAL-BCPCS, AND $n_{\mathsf{PBCPC+}}$ THE NUMBER OF CROSS-SUBTREE PARTIAL-BCPCS. UNDEFINED RATIOS ARE MARKED WITH A DASH.

Dataset	α	β	n_{MB}	n_{PBCPC}	$n_{\mathsf{PBCPC+}}$	$\frac{n_{\mathsf{PBCPC+}}}{n_{\mathsf{MB}}}$	$\frac{n_{\mathsf{PBCPC+}}}{n_{\mathsf{PBCPC}}}$
Youtube	2	2	1,769,331	292,951	48,346	2.7%	16.5%
	8	8	155,186	127,109	31,954	20.6%	25.1%
Bookcrossing	2	2	155,391	134,751	50,555	32.5%	37.5%
	8	8	0	0	0	–	–
Bibsonomy	2	2	2,132,826	899,274	277,446	13.0%	30.9%
	8	8	371,569	211,478	97,237	26.2%	46.0%
Citeseer	2	2	54,083	53,331	34,750	64.3%	65.2%
	8	8	1	1	1	100.0%	100.0%
Stackoverflow	2	2	2,922,148	1,071,784	239,306	8.2%	22.3%
	8	8	1,129	1,123	831	73.6%	74.0%
Twitter	2	2	5,102,542	2,735,473	577,820	11.3%	21.1%
	8	8	108,721	79,833	35,196	32.4%	44.1%
Imdb	2	2	1,809,175	1,354,225	495,541	27.4%	36.6%
	8	8	74,209	56,499	12,599	17.0%	22.3%
Actor2	2	2	3,761,666	2,589,905	769,313	20.5%	29.7%
	8	8	243,559	100,874	23,518	9.7%	23.3%
Amazon	2	2	5,702,211	3,407,449	648,823	11.4%	19.0%
	8	8	4,455	4,226	1,974	44.3%	46.7%
DBLP	2	2	1,281,831	1,197,005	737,028	57.5%	61.6%
	8	8	108	99	59	54.6%	59.6%
Amazon − Books	2	2	19,062,947	9,700,938	2,486,962	13.0%	25.6%
	8	8	1,630,891	1,055,673	242,926	14.9%	23.0%
Amazon − Beauty	2	2	2,998,164	2,549,447	1,304,002	43.5%	51.1%
	8	8	288	284	219	76.0%	77.1%

TABLE VI
SCALABILITY TEST OF BicliqueP (S)

	Imdb	Actor2	Amazon	DBLP
20%	3.4	1.2	8.3	0.8
40%	7.8	5.7	54.5	1.4
60%	13.5	14.7	85.2	2.8
80%	17.5	25.8	107.3	4.5
100%	18.2	49.0	111.2	5.8

TABLE VII
MEMORY USAGE (MB) OF VARIOUS ALGORITHMS

	MBAG	PBCPC	PBCPC+	Biclique	BicliqueM	BicliqueP
Imdb	**233**	364	361	260	260	343
Actor2	**409**	725	720	449	449	639
Amazon	**711**	1,228	1,207	797	772	1,130
DBLP	935	957	970	1,200	**935**	940

with four vertices: $u, w \in U_2, v, x \in V_2$, $(u, v) \in E_2, (u, x) \in E_2, (w, v) \in E_2, (w, x) \in E_2$).

In this case study, $\alpha = 4, \beta = 4, k = 12$. By Definition 4, the smallest maximal bicliques in $(4,4)$-BCPC are either $(5,4)$-bicliques or $(4,5)$-bicliques, and the edges within these maximal bicliques exist in at least $3 \times 4 = 12$ butterflies, thus, we set $k = 12$ for k-bitruss.

Figure 6 presents the two cases centered on *Xuemin Lin* and *Michael Stonebraker*. Compared with their $(4,4)$-BCPCs, the largest maximal bicliques omit important collaborators: *Kai Wang* and *Yizhang He* in the former case, and even *Michael Stonebraker* and *Samuel Madden* in the latter. In contrast, both cases belong to the same 12-bitruss and $(4,4)$-core, containing more than 3,000 papers and 300 authors and more than 17,000 papers and 5,000 authors, respectively. Relative to the BCPCs, these much larger results introduce many additional authors and papers, making the collaborators of interest difficult to analyze. This contrast illustrates the role of percolation: adjacent maximal bicliques preserve collaborators missed by any one biclique, while the resulting BCPC remains more selective than the corresponding core or bitruss.

VII. RELATED WORK

BCPC and its applications. [7] proposed the BCPC model and a detection method based on traversing the MBAG; [15] implemented this as an online toolkit. Applications include user-cluster mining in social networks [11], editor–article relationship analysis in Wikipedia [8], abnormal-behavior detection in enterprise networks [9], and student–resource pattern analysis in online learning [10], [12]. [13], [14] further studied index-based BCPC computation for repeated queries, including temporal bipartite graphs. Our previous work on k-clique percolation communities in unipartite graphs [17] used partial communities from a maximal-clique enumeration tree. Here, we adapt that high-level framework to the two-sided bipartite setting and develop a different cross-subtree search for the deeper MBE tree.

Maximal biclique and (p,q)-biclique enumeration. Maximal biclique enumeration underpins BCPC detection. The powerset-based framework of [25] was improved by pivot pruning [18], optimized vertex ordering [19], and a unified hereditary-subgraph framework [16]. More recently, [26] proposed a partition-oriented strategy with an improved worst-case time guarantee. For (p,q)-biclique enumeration, [27] uses it as a subroutine for broader analysis, while [20] focuses on optimizing the task itself; [22] addresses exact and approximate counting. Unlike stand-alone enumeration and counting, our objective is to identify the connectivity among maximal bicliques induced by shared (α,β)-bicliques.

Other cohesive subgraph models. Related models include (α,β)-core [28], [24], [29] and k-bitruss [30], [23], [31]. Maximum biclique search [32], [33], [34] and maximum edge-weighted biclique search [35] target a single optimal biclique rather than a percolation community. Modularity-based [36], [37] and label-propagation [38] methods are also relevant.

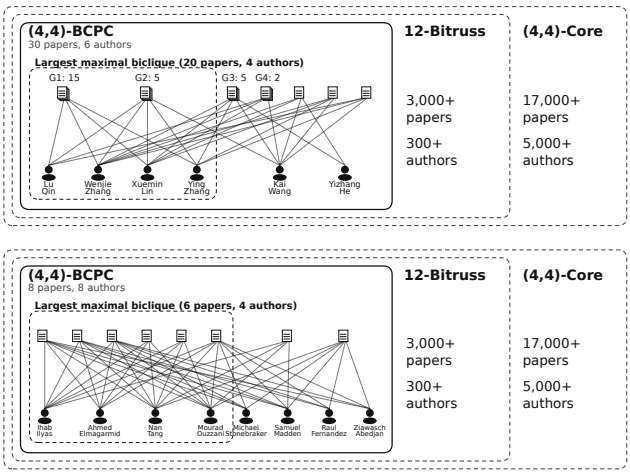


Fig. 6. Case studies on DBLP. $G_i : n$ denotes a group of n papers with the same author set within the BCPC.

VIII. Conclusion

In this work, we address BCPC detection in bipartite graphs without traversing the original massive MBAG. We adapt the high-level framework of our previous work to the two-sided (α, β) setting by computing partial-BCPCs from the MBE tree and using them to prune biclique listing. To overcome the limits of within-subtree merging, we develop a cross-subtree search that finds all maximal bicliques sharing a qualifying prefix and reuses a partial-BCPC at another qualifying node through a stop-label. Experiments on 12 real-world datasets show that this reduces the number of partial-BCPCs and accelerates both MBAG-based and listing-based solutions; our best method can be over 390× faster than MBAG. The DBLP cases show that BCPC retains important collaborators while avoiding the much larger core and bitruss results.

Acknowledgement. This work is supported by the New Generation Artificial Intelligence-National Science and Technology Major Project (2025ZD0123403) and NSFC Grants 62427808. Rong-Hua Li is the corresponding author.

IX. AI-generated Content Acknowledgement

The authors used AI-assisted tools only for language polishing and grammar checking. All research ideas, technical contributions, theoretical analyses, experimental designs, results, and conclusions are solely from the authors.

References

[1] S. Fortunato, "Community detection in graphs," *Physics reports*, vol. 486, no. 3-5, pp. 75–174, 2010.

[2] K. Steinhaeuser and N. V. Chawla, "Community detection in a large real-world social network," in *Social computing, behavioral modeling, and prediction*. Springer, 2008, pp. 168–175.

[3] S. Rahiminejad, M. R. Maurya, and S. Subramaniam, "Topological and functional comparison of community detection algorithms in biological networks," *BMC bioinformatics*, vol. 20, no. 1, p. 212, 2019.

[4] C. Choudhary, I. Singh, and M. Kumar, "Community detection algorithms for recommendation systems: techniques and metrics," *Computing*, vol. 105, no. 2, pp. 417–453, 2023.

[5] C. Maier and D. Simovici, "Bipartite graphs and recommendation systems," *Journal of Advances in Information Technology-in print*, 2022.

[6] C. Carusi and G. Bianchi, "A look at interdisciplinarity using bipartite scholar/journal networks," *Scientometrics*, vol. 122, no. 2, pp. 867–894, 2020.

[7] S. Lehmann, M. Schwartz, and L. K. Hansen, "Biclique communities," *Physical Review E—Statistical, Nonlinear, and Soft Matter Physics*, vol. 78, no. 1, p. 016108, 2008.

[8] R. Jesus, M. Schwartz, and S. Lehmann, "Bipartite networks of wikipedia's articles and authors: a meso-level approach," in *Proceedings of the 5th international symposium on Wikis and open collaboration*, 2009, pp. 1–10.

[9] Q. Liao, A. Striegel, and N. Chawla, "Visualizing graph dynamics and similarity for enterprise network security and management," in *Proceedings of the seventh international symposium on visualization for cyber security*, 2010, pp. 34–45.

[10] T. Hecking, S. Ziebarth, and H. U. Hoppe, "Analysis of dynamic resource access patterns in a blended learning course," in *Proceedings of the Fourth International Conference on Learning Analytics and Knowledge*, 2014, pp. 173–182.

[11] T. Hecking, I.-A. Chounta, and H. U. Hoppe, "Analysis of user roles and the emergence of themes in discussion forums," in *2015 Second European Network Intelligence Conference*. IEEE, 2015, pp. 114–121.

[12] S. Ziebarth, G. Neubaum, E. Kyewski, N. Kramer, H. U. Hoppe, T. Hecking, and S. Eimler, "Resource usage in online courses: Analyzing learner's active and passive participation patterns." International Society of the Learning Sciences, Inc.[ISLS]., 2015.

[13] Z. Chen, Y. Zhao, L. Yuan, X. Lin, and K. Wang, "Index-based biclique percolation communities search on bipartite graphs," in *2023 IEEE 39th International Conference on Data Engineering (ICDE)*. IEEE, 2023, pp. 2699–2712.

[14] Z. Chen, H. Huang, L. Yuan, J. Xu, B. Zheng, and X. Lin, "Biclique percolation communities computation on temporal bipartite graphs," *IEEE Transactions on Knowledge and Data Engineering*, 2026.

[15] B. Wang, J. Chen, and S. Zhang, "Bmtk: a toolkit for determining modules in biological bipartite networks," *Quantitative Biology*, vol. 6, pp. 186–192, 2018.

[16] Q. Dai, R.-H. Li, X. Ye, M. Liao, W. Zhang, and G. Wang, "Hereditary cohesive subgraphs enumeration on bipartite graphs: The power of pivot-based approaches," *Proceedings of the ACM on Management of Data*, vol. 1, no. 2, pp. 1–26, 2023.

[17] Y. Zeng, M. Qiao, R.-H. Li, H. Qin, and G. Wang, "Scaling up k-clique percolation community detection," *Proceedings of the ACM on Management of Data*, vol. 3, no. 4, pp. 1–27, 2025.

[18] A. Abidi, R. Zhou, L. Chen, and C. Liu, "Pivot-based maximal biclique enumeration." in *IJCAI*, 2020, pp. 3558–3564.

[19] L. Chen, C. Liu, R. Zhou, J. Xu, and J. Li, "Efficient maximal biclique enumeration for large sparse bipartite graphs," *Proceedings of the VLDB Endowment*, vol. 15, no. 8, pp. 1559–1571, 2022.

[20] J. Yang, Y. Peng, D. Ouyang, W. Zhang, X. Lin, and X. Zhao, "(p, q)-biclique counting and enumeration for large sparse bipartite graphs," *The VLDB Journal*, vol. 32, no. 5, pp. 1137–1161, 2023.

[21] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to algorithms*. MIT press, 2022.

[22] X. Ye, R.-H. Li, Q. Dai, H. Qin, and G. Wang, "Efficient biclique counting in large bipartite graphs," *Proceedings of the ACM on Management of Data*, vol. 1, no. 1, pp. 1–26, 2023.

[23] K. Wang, X. Lin, L. Qin, W. Zhang, and Y. Zhang, "Efficient bitruss decomposition for large-scale bipartite graphs," in *2020 IEEE 36th International Conference on Data Engineering (ICDE)*. IEEE, 2020, pp. 661–672.

[24] B. Liu, L. Yuan, X. Lin, L. Qin, W. Zhang, and J. Zhou, "Efficient (α, β)-core computation in bipartite graphs," *The VLDB Journal*, vol. 29, no. 5, pp. 1075–1099, 2020.

[25] Y. Zhang, C. A. Phillips, G. L. Rogers, E. J. Baker, E. J. Chesler, and M. A. Langston, "On finding bicliques in bipartite graphs: a novel algorithm and its application to the integration of diverse biological data types," *BMC bioinformatics*, vol. 15, no. 1, p. 110, 2014.

[26] K. Wang, K. Yu, and C. Long, "Maximal biclique enumeration with improved worst-case time complexity guarantee: A partition-oriented strategy," *Proceedings of the ACM on Management of Data*, vol. 4, no. 3 (SIGMOD, pp. 1–24, 2026.

[27] M. Mitzenmacher, J. Pachocki, R. Peng, C. Tsourakakis, and S. C. Xu, "Scalable large near-clique detection in large-scale networks via sampling," in *Proceedings of the 21th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2015, pp. 815–824.

[28] B. Liu, L. Yuan, X. Lin, L. Qin, W. Zhang, and J. Zhou, "Efficient (α, β)-core computation: An index-based approach," in *The World Wide Web Conference*, 2019, pp. 1130–1141.

[29] Q. Liu, X. Liao, X. Huang, J. Xu, and Y. Gao, "Distributed (α, β)-core decomposition over bipartite graphs," in *2023 IEEE 39th International Conference on Data Engineering (ICDE)*. IEEE, 2023, pp. 909–921.

[30] Z. Zou, "Bitruss decomposition of bipartite graphs," in *International conference on database systems for advanced applications*. Springer, 2016, pp. 218–233.

[31] F. Lin, B. Ruan, J. Gan, and L. Li, "Efficient bitruss decomposition without butterfly enumeration," in *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining V. 2*, 2025, pp. 1718–1728.

[32] B. Lyu, L. Qin, X. Lin, Y. Zhang, Z. Qian, and J. Zhou, "Maximum biclique search at billion scale," *Proceedings of the VLDB Endowment*, 2020.

[33] K. Wang, W. Zhang, X. Lin, L. Qin, and A. Zhou, "Efficient personalized maximum biclique search," in *2022 IEEE 38th International Conference on Data Engineering (ICDE)*. IEEE, 2022, pp. 498–511.

[34] Q. Dai, R.-H. Li, L. Qiao, D. Cui, and G. Wang, "Theoretically and practically efficient maximum biclique search," *Proceedings of the ACM on Management of Data*, vol. 3, no. 6, pp. 1–26, 2025.

[35] J. Wang, J. Yang, C. Zhang, and X. Lin, "Efficient maximum edge-weighted biclique search on large bipartite graphs," *IEEE Transactions on Knowledge and Data Engineering*, vol. 35, no. 8, pp. 7921–7934, 2022.

[36] M. J. Barber, "Modularity and community detection in bipartite networks," *Physical Review E—Statistical, Nonlinear, and Soft Matter Physics*, vol. 76, no. 6, p. 066102, 2007.

[37] R. Arthur, "Modularity and projection of bipartite networks," *Physica A: Statistical Mechanics and its Applications*, vol. 549, p. 124341, 2020.

[38] X. Liu and T. Murata, "An efficient algorithm for optimizing bipartite modularity in bipartite networks," *Journal of Advanced Computational Intelligence and Intelligent Informatics*, vol. 14, no. 4, pp. 408–415, 2010.